AutoLISP Builtins (Core)

Specifications

clautolisp contributors

June 24, 2026

Contents

1 Introduction

The `clautolisp.autolisp-builtins-core` system implements the AutoLISP standard library: every built-in function user code can call without `require` or `load`. This is the layer that turns the bare evaluator into something that can actually run real-world AutoLISP code.

Audience: clautolisp developers extending the builtin set, maintaining spec coverage, or chasing AutoLISP / BricsCAD / clautolisp behavioural divergences.

2 Goals

- Implement every autolisp-spec entry the runtime requires for conformance, in the four broad categories: pure-language (math, list, string, control), HAL-mediated (entity, selection-set, sysvar, command), interaction (getstring / getpoint / getreal / …), and reflection (atoms-family, type, vl-symbolp, vl-symbol-name, …).
- Surface each builtin's autolisp-spec coverage so missing / partial implementations are visible — see `builtins-coverage.md`.
- Keep host-touching builtins behind the HAL protocol so they signal :host-not-supported under NullHost but reach the in-memory fixture under MockHost.

3 Registration

Every builtin is registered via:

(make-core-builtin-subr NAME LISP-FUNCTION)

NAME is the uppercased AutoLISP symbol name (a string). The `install-core-builtins` entry point walks the entire registry list and binds each NAME's namespace cell to a fresh `autolisp-subr`. The runtime then dispatches the call through the cell's value.

`make-core-builtin-subr` wraps the Lisp function with a thin error boundary that turns CL conditions into `autolisp-runtime-error` with a builtin-name-prefixed message.

4 Argument coercion

The `require-*` family (require-int32, require-real, require-string, require-proper-list, require-int-or-real, …) is the standard type-check / coerce helper. Each variant signals `:invalid-NUMBER-argument` / `:invalid-STRING-argument` / etc. with the builtin name as context.

The coercion philosophy:

- Type errors are reported with the OFFENDING value's `(type ...)`, the EXPECTED type, and the BUILTIN name so the user sees `(SOMEFUN: expected INT, got "abc")`.
- Integer / real distinction matches AutoLISP semantics (`(/ 5 2) = 2` vs `(/ 5 2.0) = 2.5`).
- Strings flow through `autolisp-string` so the printer surfaces them with the right escape semantics.

5 Builtin categories

The file `source/api.lisp` groups builtins by autolisp-spec chapter:

- **Math** (ch.5) — `+ - * /`, `sqrt`, `sin`, `cos`, `atan`, `expt`, `log`, `fix`, `abs`, …
- **Bitwise** — `logand`, `logior`, `lsh`, `boole`, …
- **List** (ch.5/13) — `car` / `cdr` / `cons` / `list` / `mapcar` / `apply` / `foreach`-helpers / `assoc` / `subst` / `append` / `reverse` / `last` / `length` / `nth` / `member` / `position`, …
- **String / conversion** (ch.7,11) — `strcat` / `substr` / `strlen` / `strcase` / `read` / `read-line` / `atoi` / `atof` / `itoa` / `rtos` / `angtos` / `vl-string-*` `family`, …
- **Geometry** — `distance` / `angle` / `polar` / `trans` / `inters`, …
- **Predicate / reflection** — `type` / `null` / `not` / `atom` / `vl-symbolp` / `vl-symbol-name` / `vl-symbol-value` / `atoms-family`, …
- **Tracing / help stubs** — `setfunhelp` / `trace` / `untrace` (recorded but inert at runtime).
- **Entity** (ch.16, Phase 10) — HAL-mediated `entget` / `entmod` / `entmake` / `entdel` / …
- **Selection-set** (Phase 11) — `ssget` / `sslength` / `ssname` / `ssadd` / `ssdel` / …
- **Table / sysvar** (Phase 11) — `tblsearch` / `tblnext` / `getvar` / `setvar` / …
- **Interaction** (Phase 12) — `getstring` / `getpoint` / `getreal` / `getint` / `getkword` / `getcorner` / `getorient` / `getdist`, …

6 HAL-mediated builtins

Builtins that touch the host call `current-evaluation-host` to resolve the active session's backend, then delegate to a generic function from `clautolisp.autolisp-host`. Under NullHost every host call signals `:host-not-supported`; under MockHost they reach the in-memory methods.

This split lets pure-language tests run without a HAL attached and keeps the builtin layer agnostic of which backend the engine ended up driving.

7 Spec coverage tracking

`tools/builtins-coverage/coverage.sh` produces `documentation/builtins-coverage.md`: a per-symbol roll-up of the autolisp-spec entries vs. the registered builtins. The output is the audit material a developer uses to plan a coverage push.

Open gaps are tracked in `builtins-coverage.md`'s "Not implemented" section and become individual issues when prioritised.

8 Test coverage

Tests live under `tests/`. Naming follows the spec chapter:

- `math-tests.lisp`, `bitwise-tests.lisp`, `list-tests.lisp`, `string-tests.lisp`, `conversion-tests.lisp`, `geometry-tests.lisp`, `predicate-tests.lisp`, `reflection-tests.lisp` — pure-language coverage.
- `entity-tests.lisp`, `sysvar-tests.lisp`, `selection-set-tests.lisp`, `interaction-tests.lisp` — HAL-mediated.
- `coverage-tests.lisp` — asserts the registry has every builtin the spec says ships.

The suite runs against the runtime + MockHost; under NullHost the HAL-mediated tests skip with a recorded marker.

9 References

- Source: `source/api.lisp` (the registration table + per-builtin CL functions).
- Tests: `tests/`.

- Sibling track: `autolisp-builtins-core-user-manual.org`.
- Spec: `autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org`.
- Coverage: `documentation/builtins-coverage.md`.

10 Synchronisation

Per AGENTS.md, changes to the code / this spec / the user-manual must be propagated. The HAL protocol changes also propagate to `clautolisp.autolisp-host` specs.